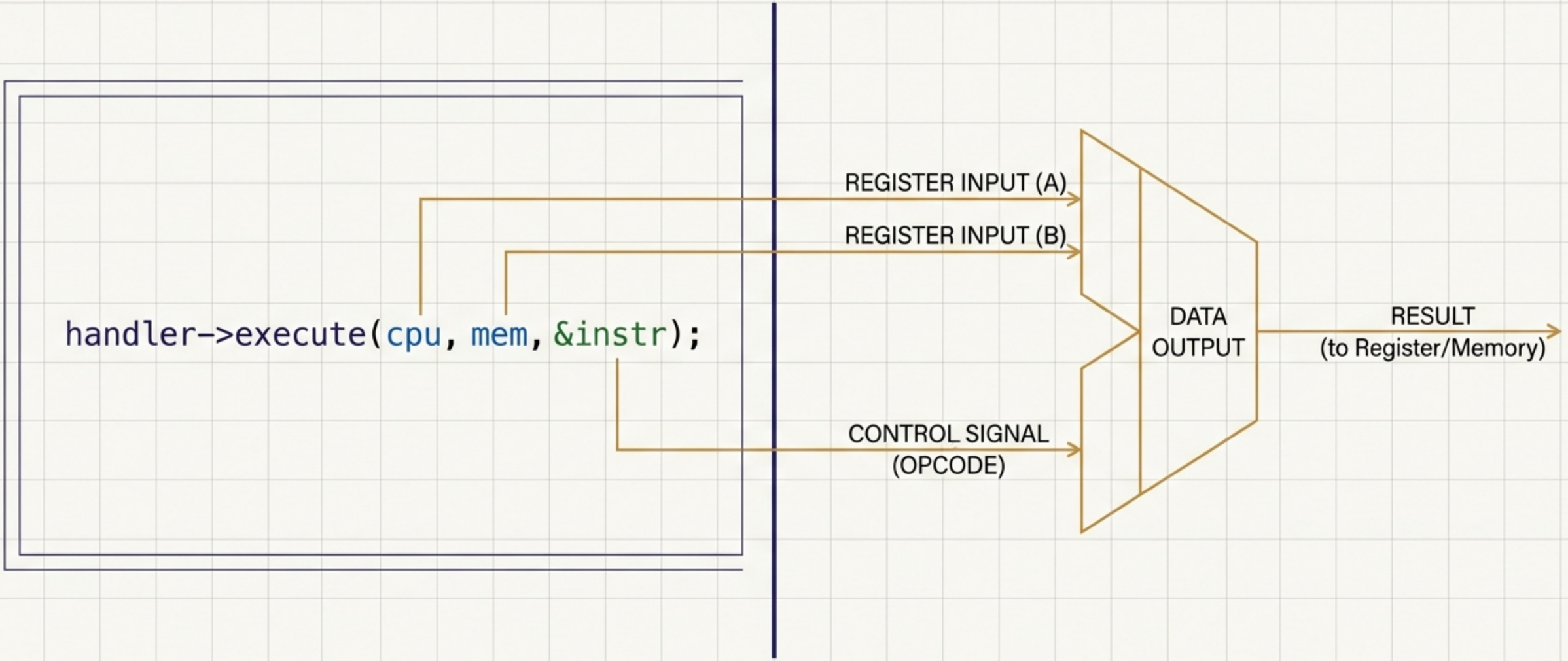
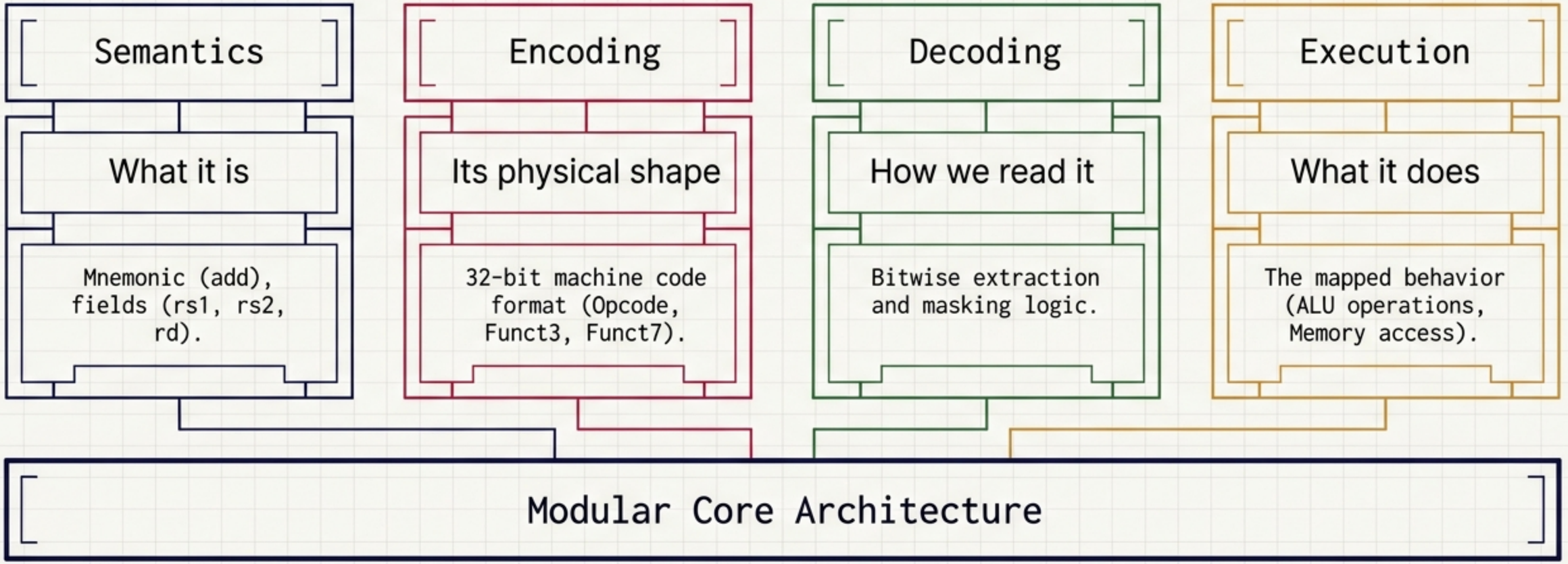


Software as Silicon: Demystifying the RV32I Processor Simulator

Using C programming to model architectural state, dynamic dispatch, and the RISC-V execution cycle.

Gemini Notebook

The Core Philosophy: Modularity



The brilliance of this codebase is its modularity. By separating what an instruction looks like from how it executes, the code remains clean, extensible, and perfect for teaching.

Gemini Notebook

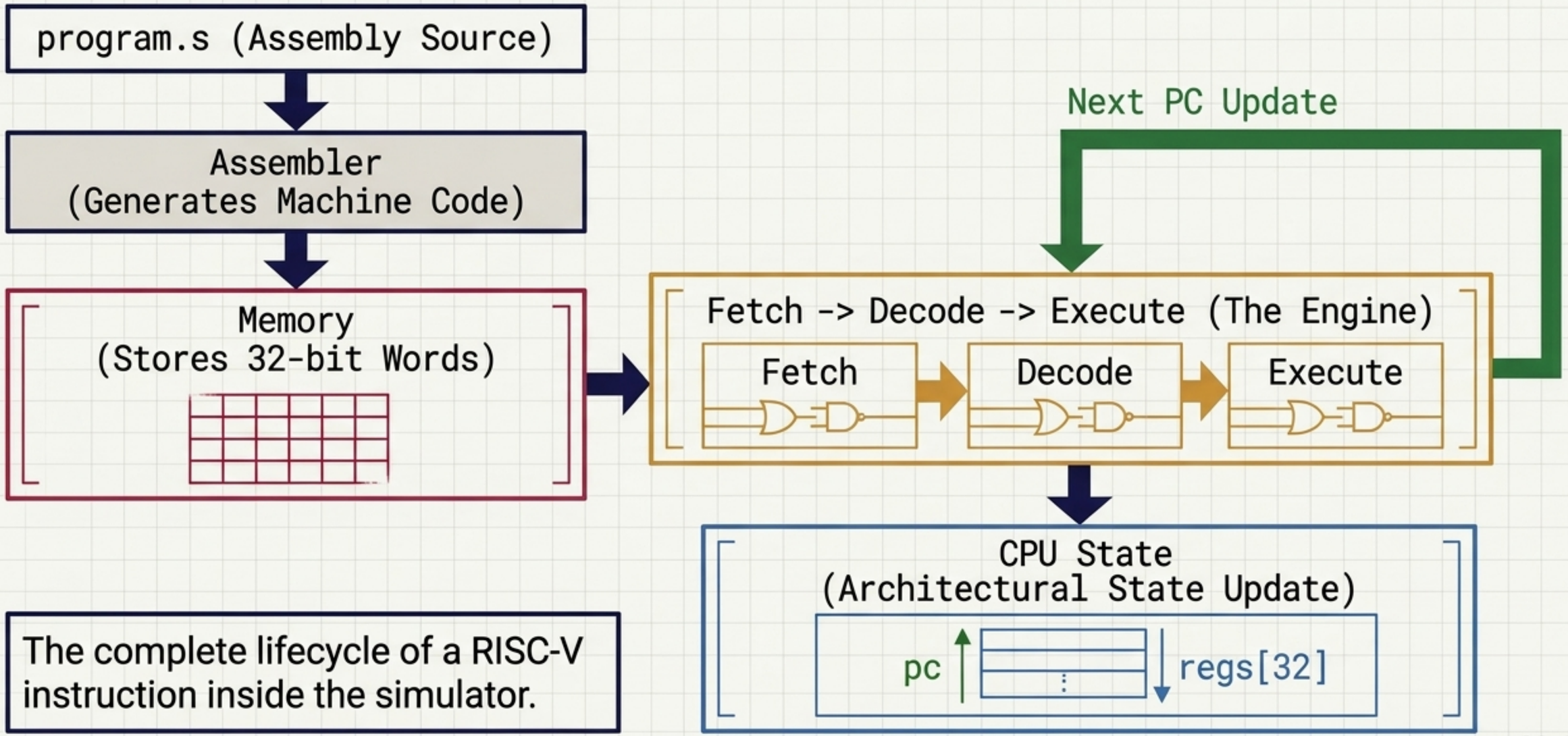
The Software-to-Hardware Translation Matrix

C Codebase Component	Processor Concept
`CPUState struct`	Architectural State
`regs[32]` / `pc`	Register File & Program Counter
`Memory struct`	Main Memory
`decoder.c`	Instruction Decoder
`isa_registry.c`	Instruction Dispatch
`Function Pointers`	Instruction Control
`rv32i_base.c`	ISA Execution Logic
`execute_cycle()`	Processor Cycle

A cheat sheet for the architecture. Every file and struct in this C project has a physical hardware counterpart.

Gemini Notebook

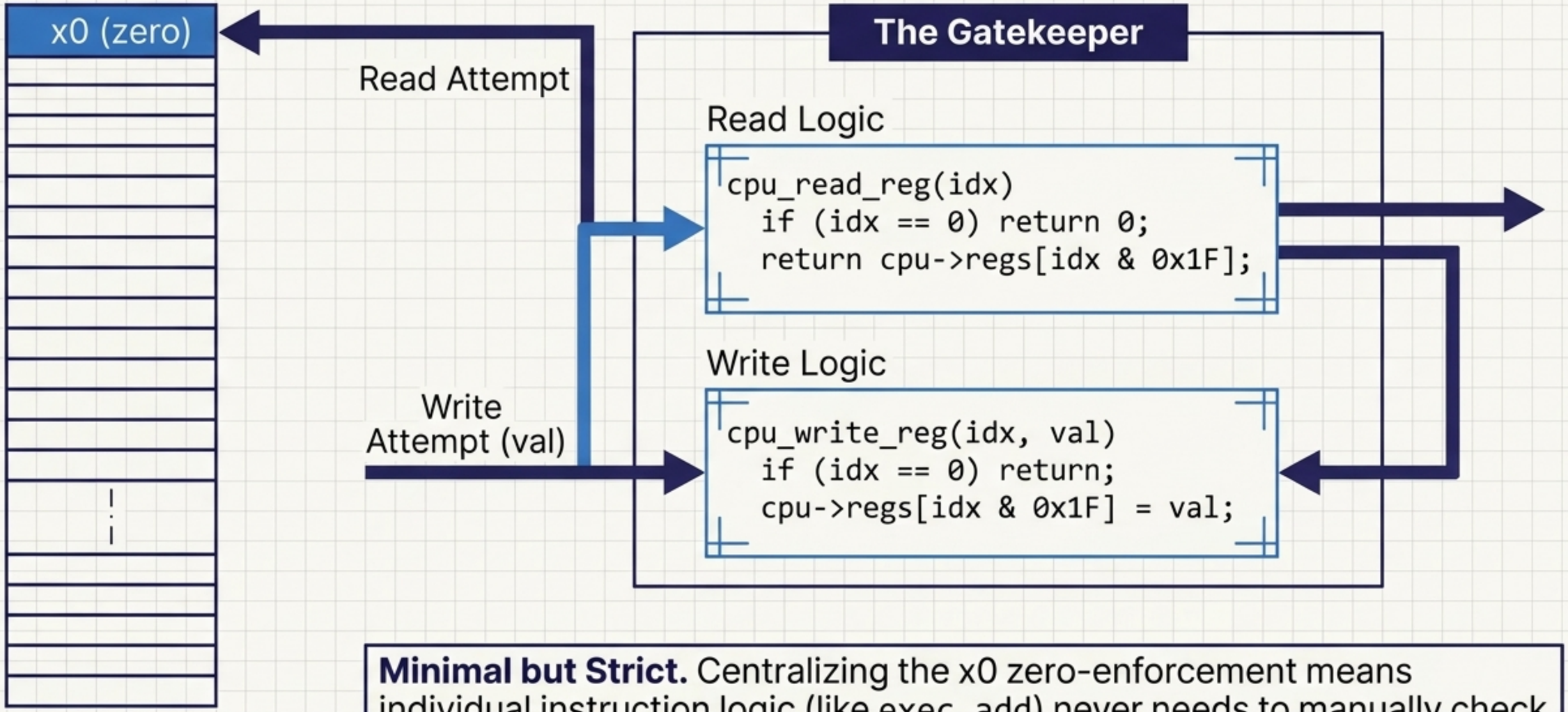
The Macro Architecture & Instruction Lifecycle



The complete lifecycle of a RISC-V instruction inside the simulator.

Gemini Notebook

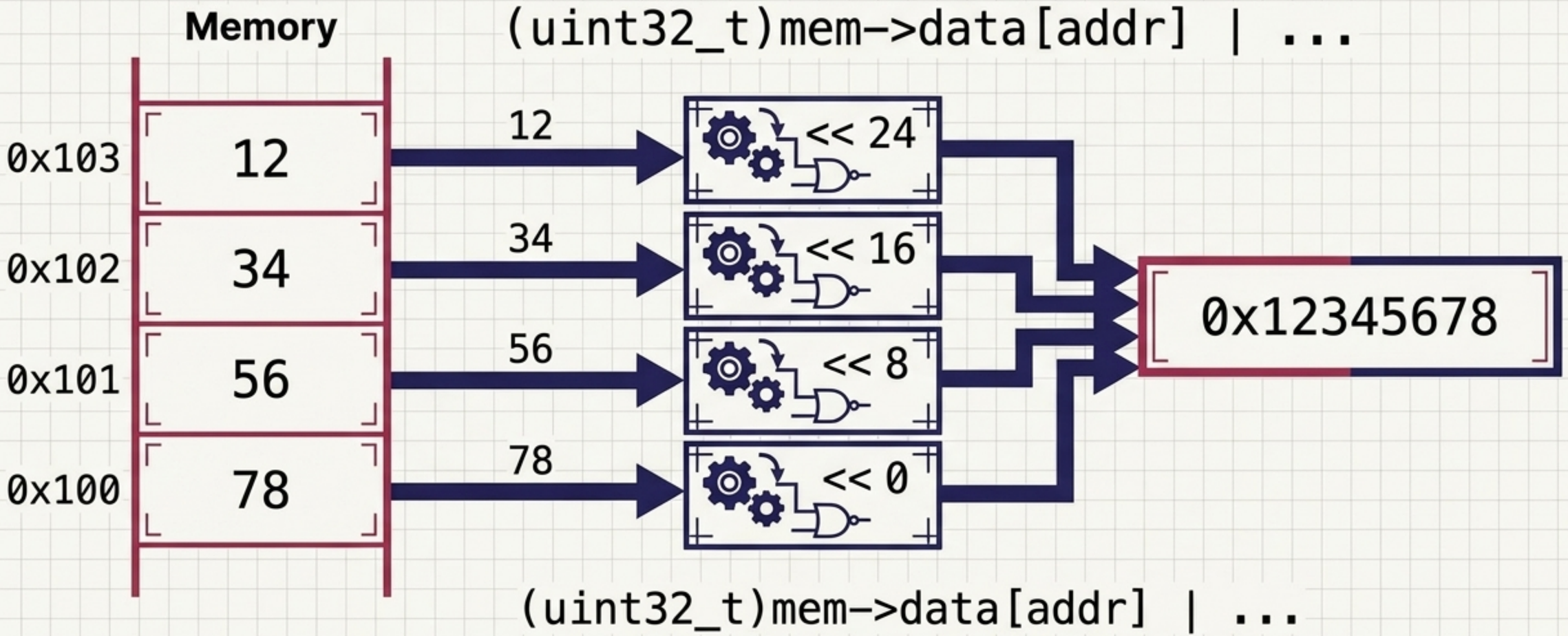
Anatomy of the CPU State



Minimal but Strict. Centralizing the x0 zero-enforcement means individual instruction logic (like exec_add) never needs to manually check if it is writing to the zero register.

Gemini Notebook

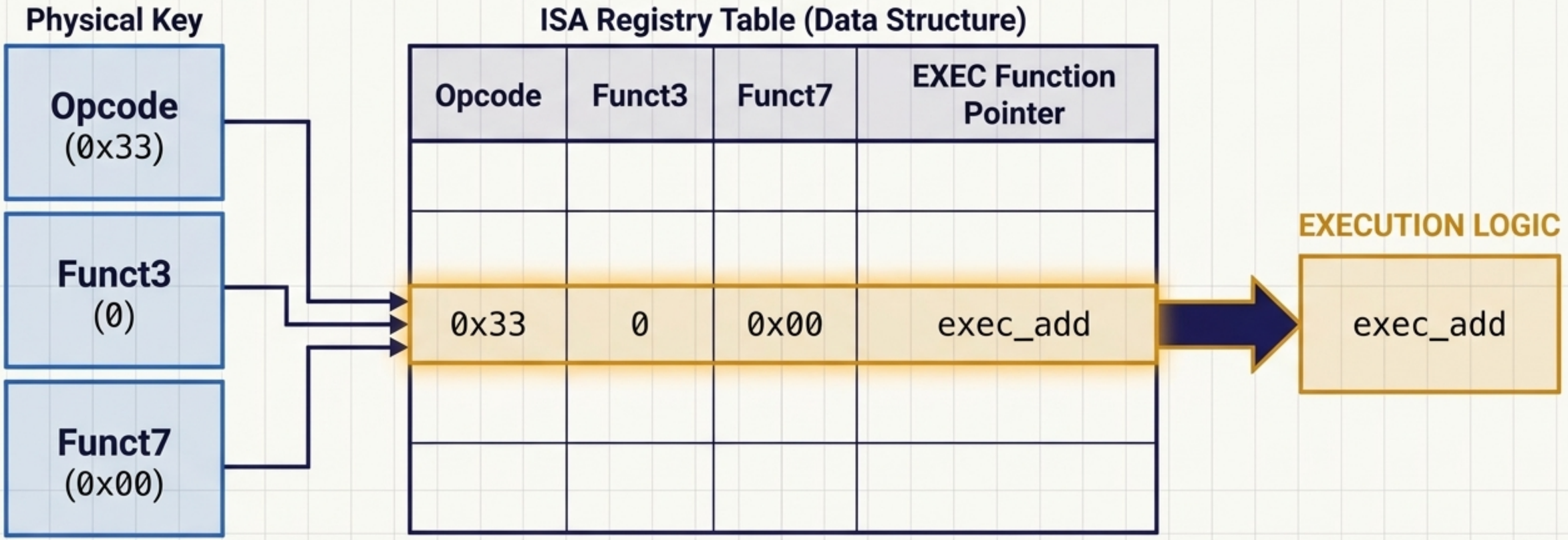
Memory & Little-Endian Reconstruction



Simulated memory is a simple byte array, but reconstructing 32-bit words requires explicit bitwise logic to simulate little-endian architecture natively and avoid C aliasing issues.

Gemini Notebook

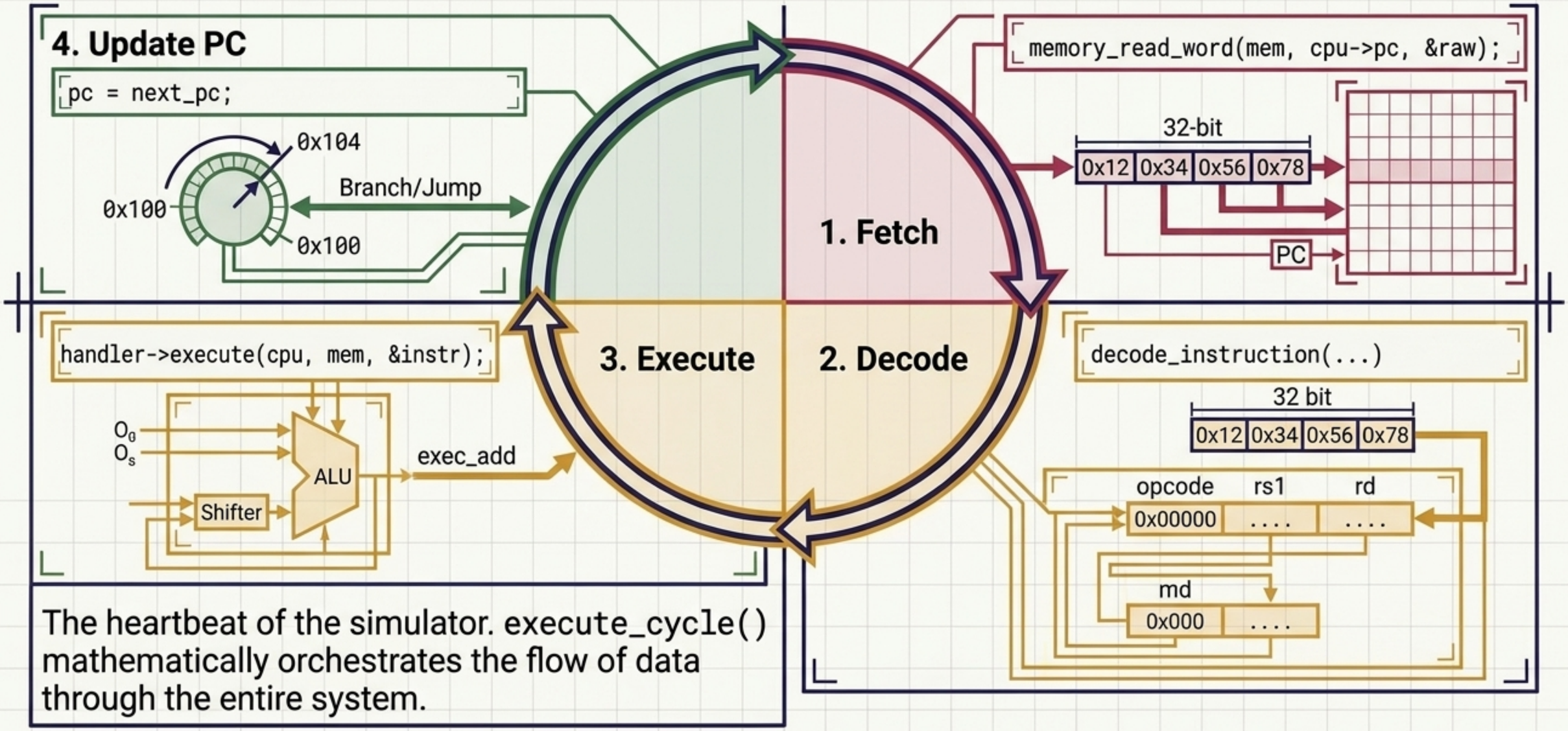
The ISA Registry: The Engine's Brain



The ISA Registry treats instruction definitions as a data table mapped to C function pointers. This elegantly replicates hardware dynamic dispatch and makes adding new ISA extensions trivial by avoiding messy switch statements.

Gemini Notebook

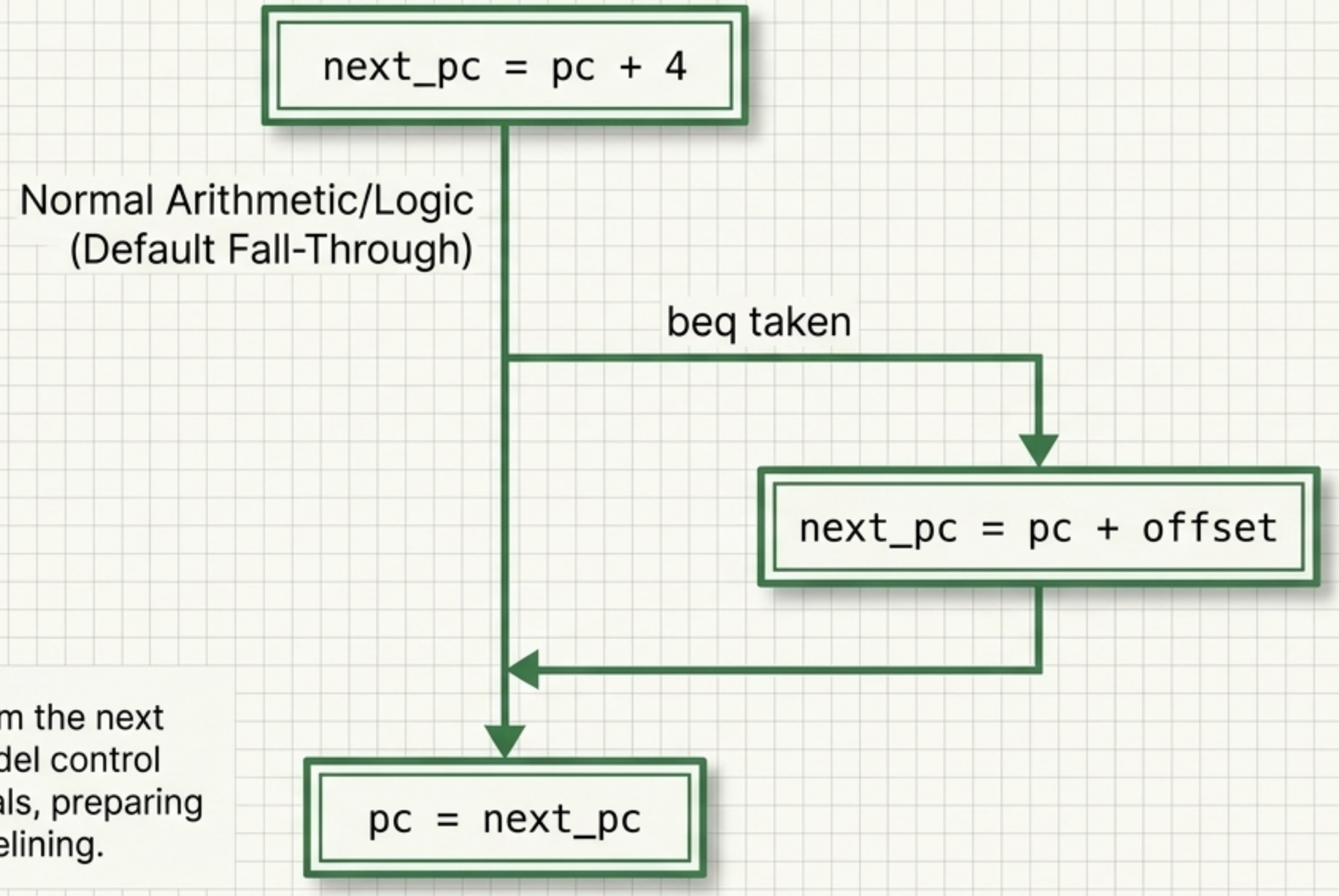
The 4-Stage Execute Cycle



The heartbeat of the simulator. `execute_cycle()` mathematically orchestrates the flow of data through the entire system.

Gemini Notebook

Elegant Control Flow: The next_pc Logic

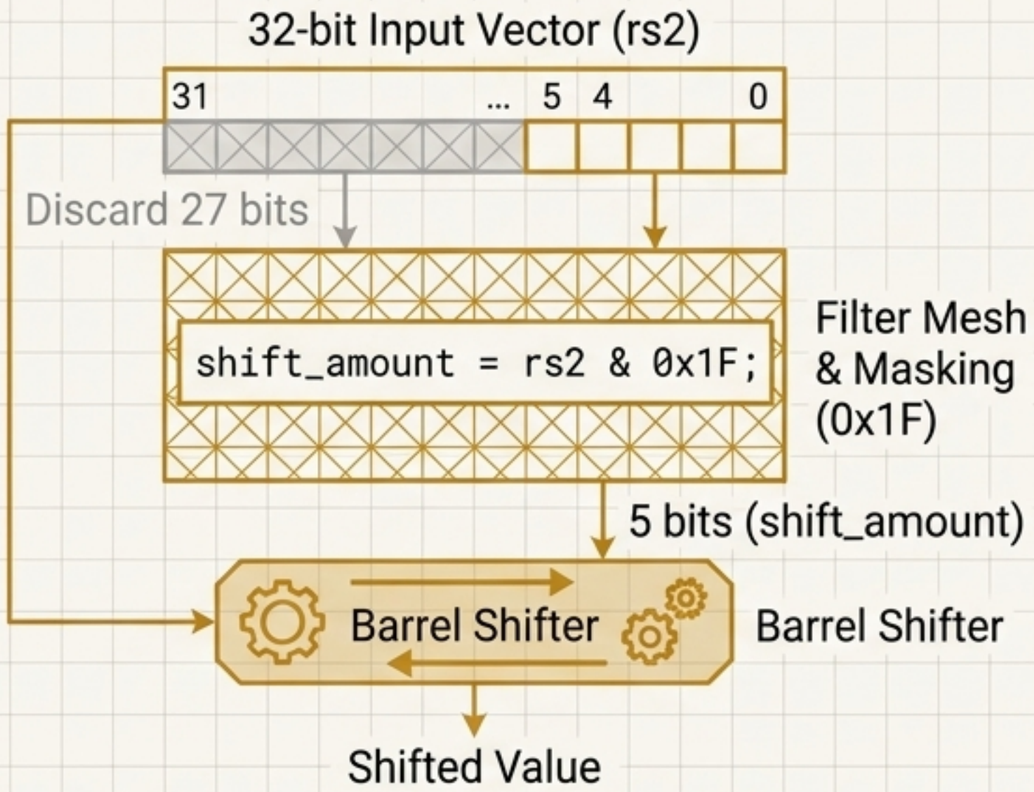


Decoupling the current PC from the next PC creates a clean way to model control flow without messy conditionals, preparing the architecture for future pipelining.

Gemini Notebook

Instruction Deep Dive: Arithmetic & Logic

Simulating Hardware Barrel Shifters



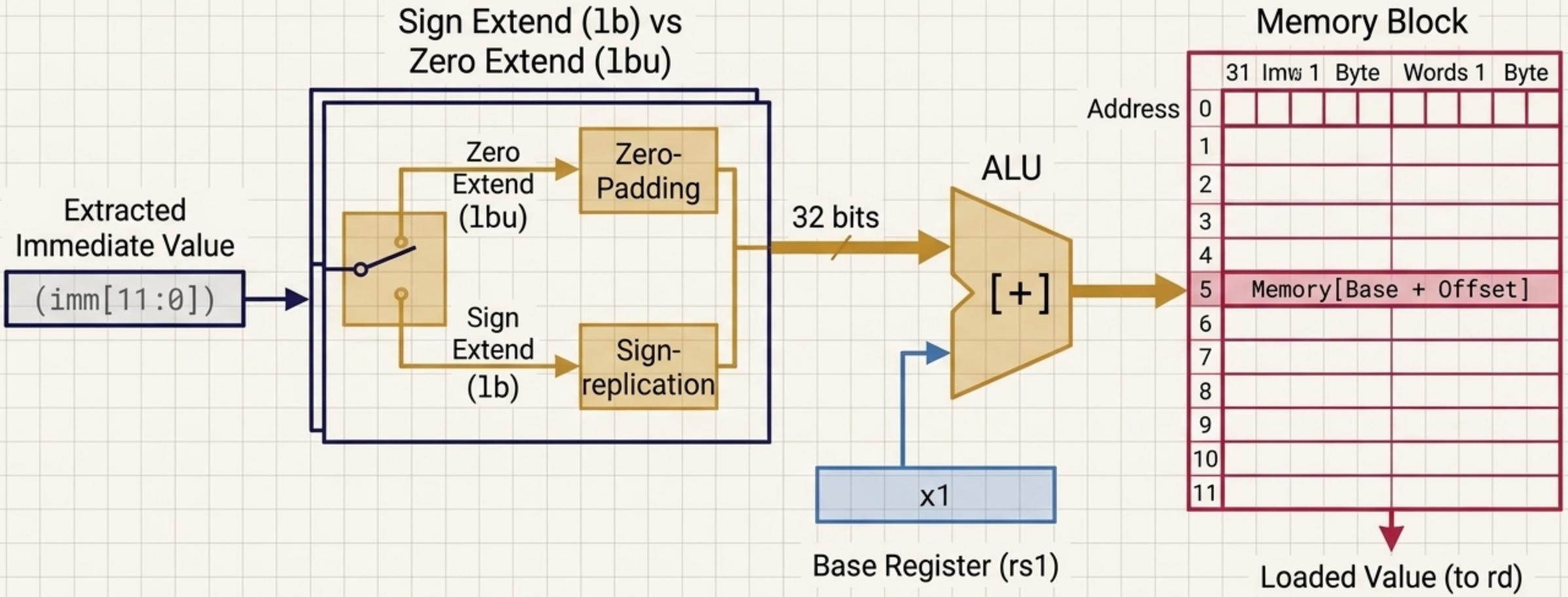
Signed vs. Unsigned Semantics

slt (Signed Less Than)
`int32_t (-1) < 1 -> True`
sltu (Unsigned Less Than)
`uint32_t (4294967295) < 1 -> False`

The C native type system and **bitwise operators** (`int32_t` vs `uint32_t`) are beautifully leveraged as **literal hardware** mechanisms to natively model complex ISA semantics.

Gemini Notebook

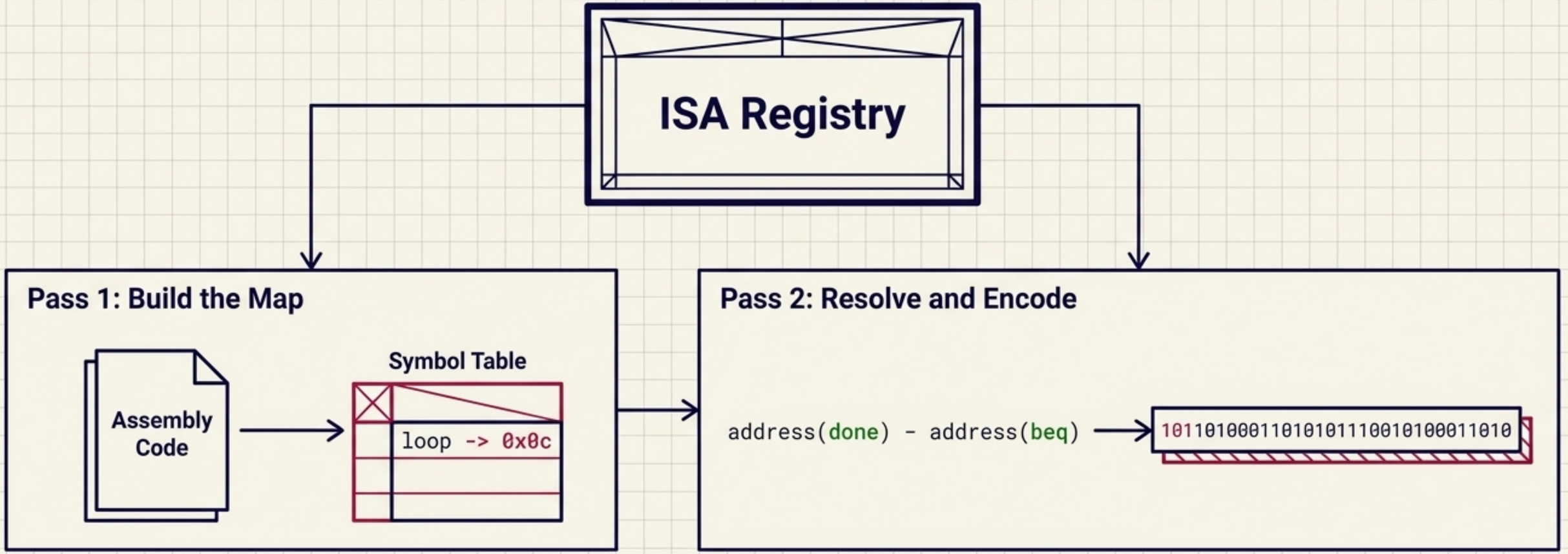
Instruction Deep Dive: Memory & Addresses



For memory calculations, the decoder does the heavy lifting of preparing and sign-extending immediate values exactly like a **physical hardware decode stage**, before it reaches execution.

Gemini Notebook

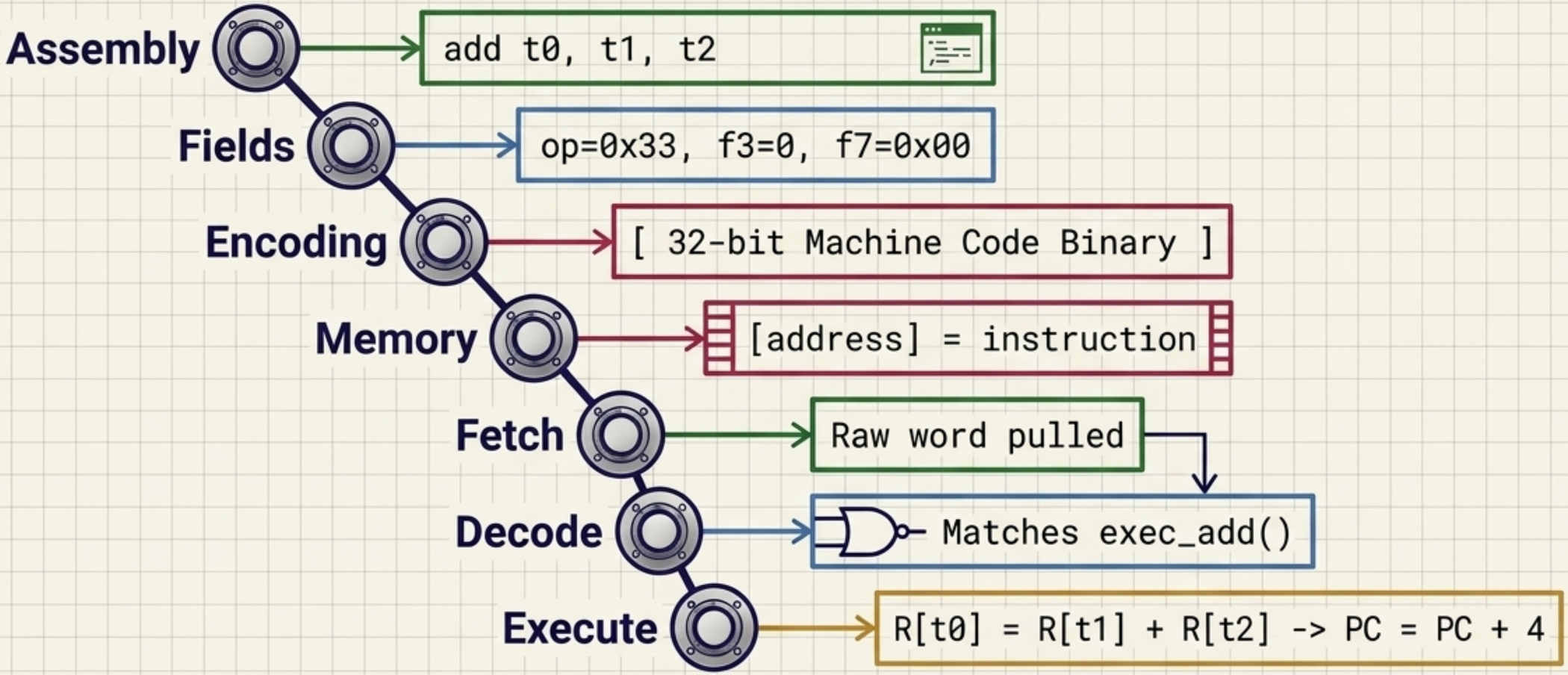
The Assembler Toolchain



The assembler requires two passes to resolve forward-referencing labels. It relies on the exact same ISA registry as the decoder, establishing a single unified source of truth for the entire toolchain.

Gemini Notebook

Tracing a Single Instruction



Putting it all together. A single instruction's journey from human-readable text to a permanent change in architectural state.

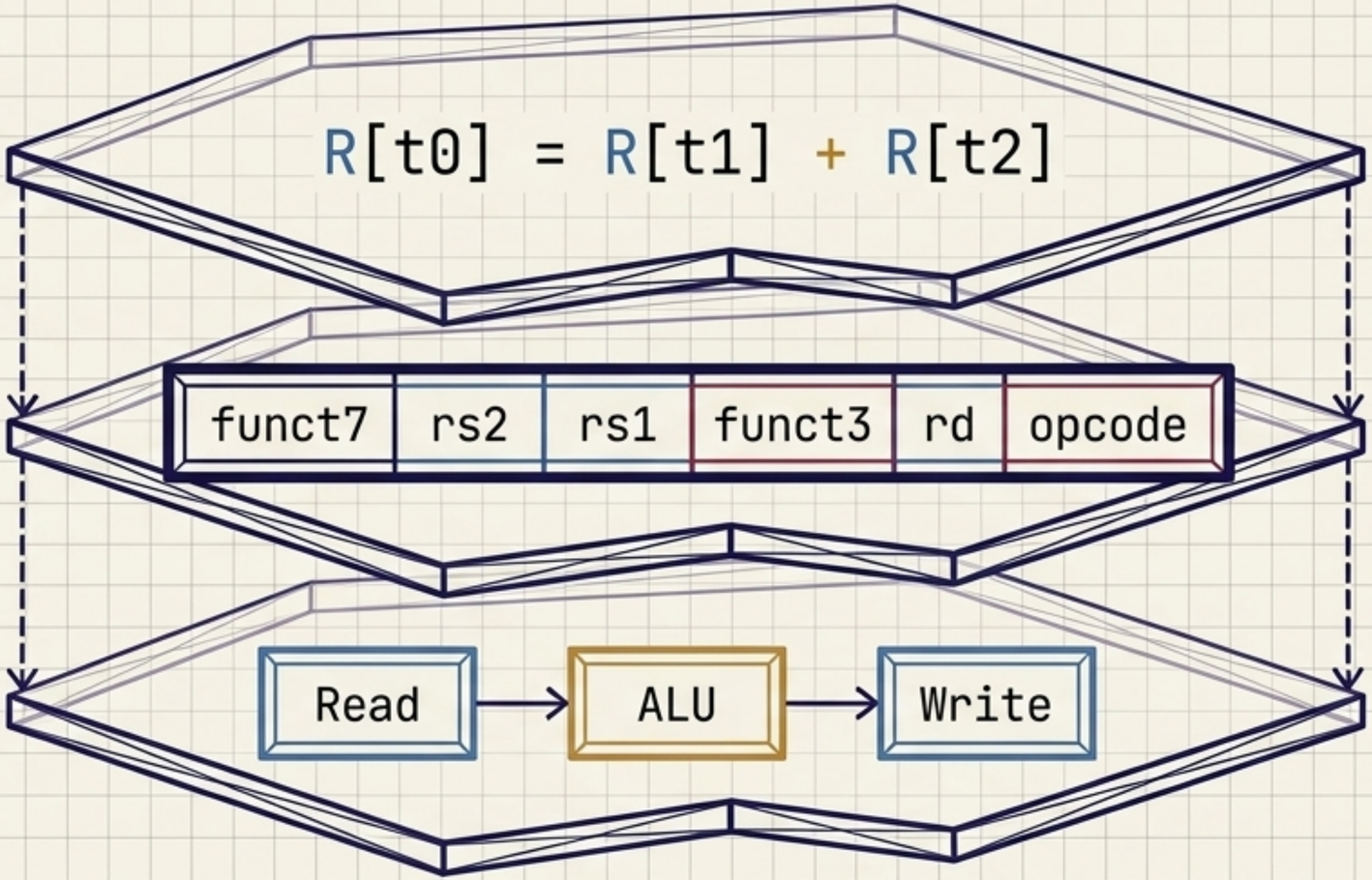
Gemini Notebook

Synthesis: The Three Layers of Understanding

Semantics
(The Meaning)

Encoding
(The Shape)

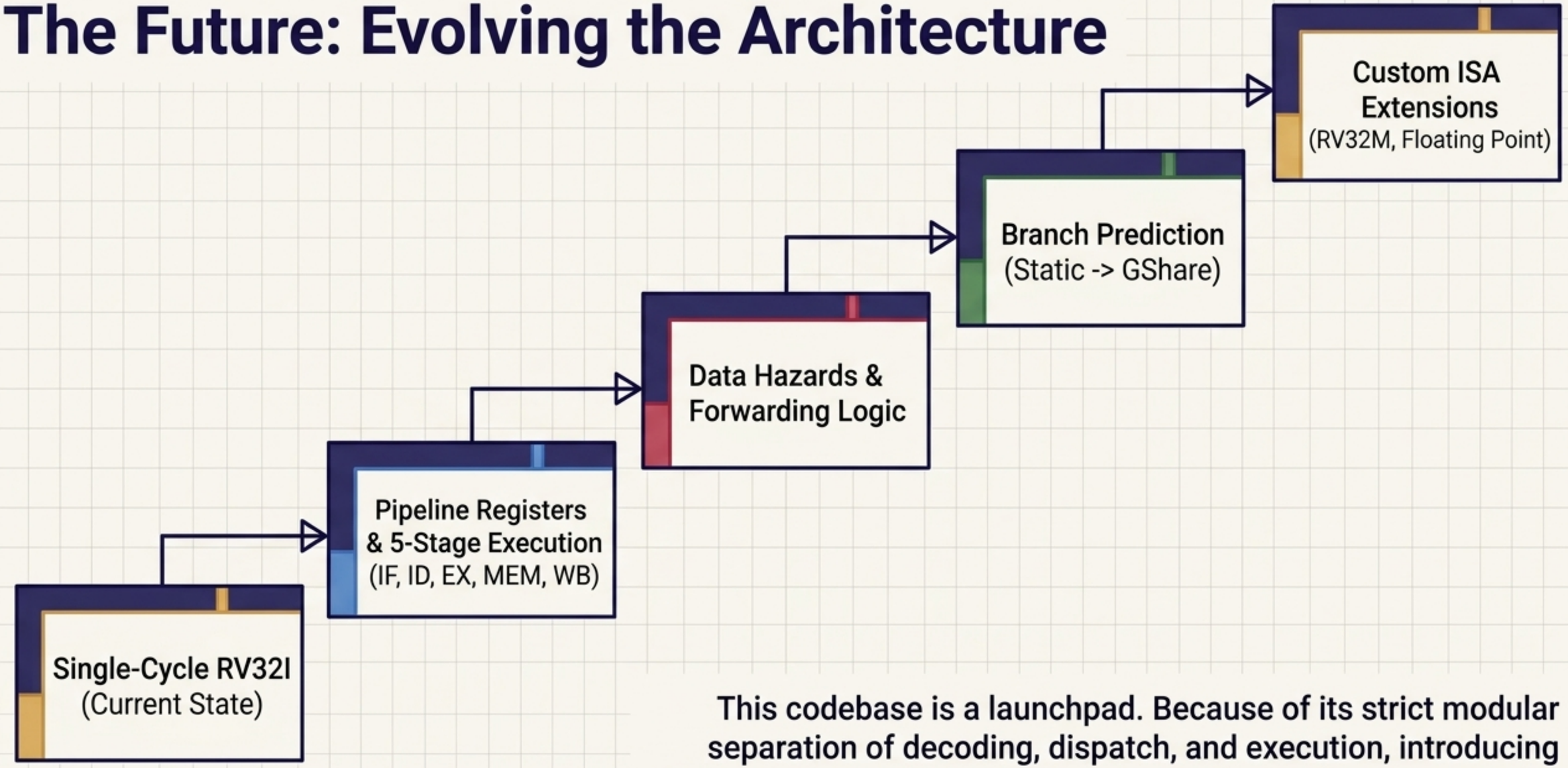
Execution
(The Mechanism)



To master computer architecture, you must never mix these up. Understand an instruction's meaning, its shape, and its mechanical execution as distinct but seamlessly connected realities.

Gemini Notebook

The Future: Evolving the Architecture



This codebase is a launchpad. Because of its strict modular separation of decoding, dispatch, and execution, introducing pipeline registers and hazards is a natural, seamless progression.

Gemini Notebook